

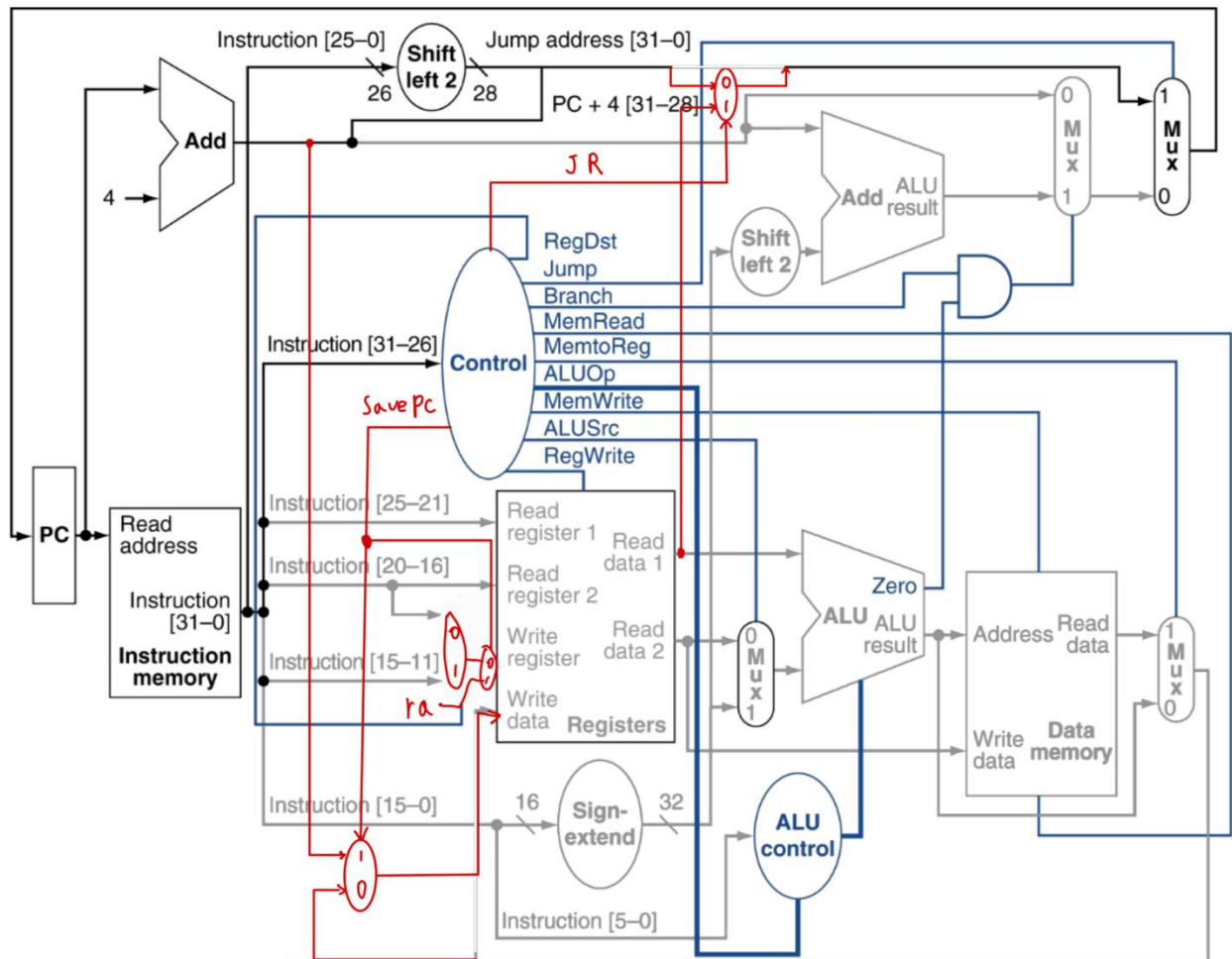
Lab3 Report

Name: 박수현

Student ID: 2022063672

Class Code: 12916

1. Overall Structure



위 그림은 single cycle CPU의 전체적인 구조를 나타낸 그림이다. 붉은 선으로 표시된 부분이 JR과 JAL 명령어를 위해 추가된 부분이다. JR명령어는 rs레지스터에 저장되어 있는 값을 그대로 PC값으로 설정하는 간단한 명령어다. 구조도의 우측 상단의 붉은 선들이 이 명령어를 구현하기 위해 추가된 부분이다. Read data1 출력값을 $\text{Jump} = 1, \text{JR} = 1$ 인 상황에서 PC로 전달해 주면 되므로, JR 컨트롤 비트를 받아 출력값을 연결시켜주는 2:1 MUX를 Jump 목적지가 전달되는 부분에 추가하여 구현할 수 있었다.

JAL 명령어는 J 명령어와 같은 동작을 하지만 PC+4의 값을 ra 레지스터에 저장하는 동작이 추가된 명령어다. 강의자료에서 제시된 구조도에는 이미 J 명령어를 위한 회로가 구현되어 있으므로, PC+4의 값을 지정된 레지스터에 저장하는 동작만 구현하면 된다. 위 구조도의 좌측 하단이 JAL 명령어를 위해 추가된 부분이다. Adder에서 출력된 PC+4의 값을 2:1 MUX에 연결하고, MUX의 출력은 SavePC 컨트롤 비트를 받아 레지스터 파일에 write data에 연결시켜 준다. 그와 함께 write register 값에 ra 레지스터의 번호(31)를 입력해 주어야 한다. 이 부분은 기존 구조도에서 write register 입력 부분에 2:1 MUX를 추가하고, SavePC의 값이 1인 경우 31을 출력해 주는 방식으로 구현하였다.

이 구조도는 Single cycle CPU의 전체적인 구조를 보기 위해 작성된 것이므로, SignExtend 컨트롤 비트와 ALU control 모듈의 부재와 같은 차이점들이 존재한다.

2. Implementation

2.1. CTRL.v

CTRL.v 모듈은 CPU의 전체적인 동작을 컨트롤하는 모듈이다. 6비트의 opcode와 funct를 입력으로 받으며, 12개의 컨트롤 비트를 출력한다. 대부분의 구현은 강의자료 04와 과제 명세에 제시된 자료를 참고하여 어렵지 않게 구현할 수 있었다. 다만 R 타입 명령어는 연산의 종류가 funct에 저장되어 있다는 점을 유의하며 구현해야 했다. 그 외에도 ANDI와 같은 비트 연산은 SignExtend의 값을 0으로 만들어야 하는 등, 제대로 확인해야 하는 부분들이 있었다.

```

31  always @(*) begin
32      // FIXME
33      // 강의자료 04 - 38p
34      RegDst = opcode == 0;
35      ALUSrc = (opcode != 0) && (opcode != `OP_BEQ) && (opcode != `OP_BNE);
36      MemtoReg = opcode == `OP_LW;
37      RegWrite = (opcode != `OP_SW) && (opcode != `OP_BEQ) && (opcode != `OP_BNE) && (opcode != `OP_J) && !((opcode == 0)
38      MemRead  = (opcode == `OP_LW);
39      MemWrite = (opcode == `OP_SW);
40      Jump     = (opcode == `OP_J) || (opcode == `OP_JAL) || (opcode == 0 && funct == `FUNCT_JR);
41      Branch   = (opcode == `OP_BEQ) || (opcode == `OP_BNE); // branch 컨디션 확인은 다른 코드에서?
42
43      JR = opcode == 0 && funct == `FUNCT_JR;
44      // troubleshooting
45      SignExtend = (opcode != `OP_ANDI) && (opcode != `OP_ORI) && (opcode != `OP_XORI);
46      // SignExtend = 1;
47      SavePC = opcode == `OP_JAL;
48

```

```

49      // ALUOp
50      if(opcode == 0) begin
51          case (funct)
52              `FUNCT_SLL: ALUOp = `ALU_SLL;
53              `FUNCT_SRL: ALUOp = `ALU_SRL;
54              `FUNCT_SRA: ALUOp = `ALU_SRA;
55              // `FUNCT_JR: ALUOp = ;
56              `FUNCT_ADDU: ALUOp = `ALU_ADDU;
57              `FUNCT_SUBU: ALUOp = `ALU_SUBU;
58              `FUNCT_AND: ALUOp = `ALU_AND;
59              `FUNCT_OR: ALUOp = `ALU_OR;
60              `FUNCT_XOR: ALUOp = `ALU_XOR;
61              `FUNCT_NOR: ALUOp = `ALU_NOR;
62              `FUNCT_SLT: ALUOp = `ALU_SLT;
63              `FUNCT_SLTU: ALUOp = `ALU_SLTU;
64          endcase
65      end
66      else begin
67          case (opcode)
68              // `OP_J: ALUOp = ;
69              // `OP_JAL: ALUOp = ;
70              `OP_BEQ: ALUOp = `ALU_EQ;
71              `OP_BNE: ALUOp = `ALU_NEQ;
72              `OP_ADDIU: ALUOp = `ALU_ADDU;
73              `OP_SLTI: ALUOp = `ALU_SLT;
74              `OP_SLTIU: ALUOp = `ALU_SLTU;
75              `OP_ANDI: ALUOp = `ALU_AND;
76              `OP_ORI: ALUOp = `ALU_OR;
77              `OP_XORI: ALUOp = `ALU_XOR;
78              `OP_LUI: ALUOp = `ALU_LUI;
79              `OP_LW: ALUOp = `ALU_ADDU;

```

2.2. CPU.v

CPU.v 모듈은 CPU 내부에 4가지 모듈을 연결하고, 간단한 shift, add 연산과 입력 지점의 MUX 연산이 구현되어 있다. 코드 작성 전에 미리 구조도를 작성해 두었으므로, wire 변수들을 구조도에 맞게 할당하거나 적절한 연산 코드를 작성하면 되었다. 대부분의 변수들이 wire로 선언되었기 때문에 assign 구문을 통해 할당을 진행하였다. MUX의 경우에는 $A ? B : C$ 형태의 삼항연산자를 이용해 구현했다.

PC_next의 값을 결정하는 코드는 가독성을 위해 if 구문으로 작성했다.

```
68 // 할당 시작
69
70 // inst;
71 assign opcode = inst[31:26];
72 assign rs = inst[25:21];
73 assign rt = inst[20:16];
74 assign rd = inst[15:11];
75 assign shamt = inst[10:6];
76 assign funct = inst[5:0];
77 assign immi = inst[15:0];
78 assign immj = inst[25:0];
79
80 // Sign extend the immediate
81 // trouble shooting
82 assign ext_imm = SignExtend ? {{16{immi[15]}}, immi} : {16'b0, immi};
83
84 // RF-related wires4
85 assign rd_addr1 = rs;
86 assign rd_addr2 = rt;
87
88 // MEM-related wires
89 assign mem_addr = alu_result;
90 assign mem_write_data = rd_data2;
91
92 // ALU-related wires
93 assign operand1 = rd_data1;
94 assign operand2 = ALUSrc ? ext_imm : rd_data2;
```

```
98 always @(*) begin
99     wr_addr = SavePC ? 5'b11111 : (RegDst ? rd : rt);
100     wr_data = SavePC ? PC+4 : (MemtoReg ? mem_read_data : alu_result);
101
102     // Define PC
103     // PC;
104     // PC_next;
105
106     if(Jump)begin
107         if(JR) begin //trouble shooting
108             PC_next = rd_data1;
109         end
110         else begin
111             PC_next = {{PC[31:28]}, immj, 2'b00};
112         end
113     end
114     else begin
115         if(Branch && alu_result) begin // 이 부분 확인 필요
116             PC_next = PC + 4 + (ext_imm << 2);
117         end
118         else begin
119             PC_next = PC + 4;
120         end
121     end
122 end
123 end
124
```

3. Troubleshooting

3.1. SignExtension

```
44 // troubleshooting
45 SignExtend = (opcode != `OP_ANDI) && (opcode != `OP_ORI) && (opcode != `OP_XORI);
46 // SignExtend = 1;
```

```
80 // Sign extend the immediate
81 // trouble shooting
82 assign ext_imm = SignExtend ? {{16{immi[15]}}, immi} : {16'b0, immi};
83
```

과제를 시작하면서 스켈레톤 코드를 구조도와 대조하며 보던 중, 구조도의 Control 모듈에는 나와있지 않던 SignExtend 출력값이 있는 것을 발견하였다. 구조도에는 sign extend가 항상 시행되는 것으로 나와있어 처음에는 항상 1이 출력되도록 코드를 작성했다. 코드는 당연히 테스트를 통과하지 못했고, 과제 명세를 천천히 다시 읽어보며 이 값이 추가된 의도를 이해했다.

또한 verilog의 문법이 익숙하지 않아, 16비트의 immi 값을 32비트로 늘리는 과정을 단순히 "immi << 16" 과 같이 작성했다. 이 문제는 verilog의 비트 결합 연산자를 통해 해결했다.

3.2. Small mistakes in CTRL module

```
40 Jump = (opcode == `OP_J) || (opcode == `OP_JAL) || (opcode == 0 && funct == `FUNCT_JR);
41 Branch = (opcode == `OP_BEQ) || (opcode == `OP_BNE); // branch 컨디션 확인은 다른 코드에서?
```

CTRL.v 를 작성하는 과정은 어렵지는 않지만 신경써야 할 부분이 많아 작은 실수들이 많이 나왔다. 예를 들어, Jump 출력값을 계산하는 과정에서 opcode가 0이라는 것을 생각하지 못하고 opcode == `FUNCT_JR; 와 같이 작성하는 등의 여러 작은 실수가 있었다.

이 부분은 과제 명세에 제시된 MIPS 명령어 명세와 강의자료 04에 제시된 표를 참고하여 해결하였다.